

第3次实验：读程序写结果

第1题：if-elif-else 只执行第一个满足条件的分支

以下程序的输出结果是：

```
a = 30
b = 1
if a >= 10:
    a = 20
elif a >= 20:
    a = 30
elif a >= 30:
    b = a
else:
    b = 0
print('a={}, b={}'.format(a, b))
```

参考答案：a=20, b=1

解析：

1. 程序开始时 $a = 30$, $b = 1$ 。接着进入条件结构。
2. 第一个条件 $a \geq 10$ 成立，因为 30 大于等于 10，所以立即执行该分支，把 a 改为 20。
3. 在 if-elif-elif-else 结构中，只要前面的一个分支已经执行，后面的 elif 和 else 都不会再判断。因此 b 一直保持原来的 1。elif 是 else if 的缩写表示否则如果就，在 if 条件失败时再判断 elif 的条件
4. 最后用 `format(a, b)` 把两个变量填入字符串，所以输出 $a=20$, $b=1$ 。

第2题：双重循环中 break 只跳出内层循环

以下程序的输出结果是：

```
for i in range(3):
    for s in "abcd":
        if s == "c":
            break
        print(s, end="")
```

参考答案：ababab

解析：

1. 外层循环 `range(3)` 会循环 3 次。每一次外层循环开始时，内层都会重新遍历字符串 `abcd`。
2. 内层循环中，当 `s` 为 `a` 时，不满足 `s == "c"`，输出 `a`；当 `s` 为 `b` 时，也不满足，输出 `b`。
3. 当 `s` 到 `c` 时，条件成立，执行 `break`，只跳出当前内层循环，字符 `c` 和后面的 `d` 都不会输出。
4. 所以每轮外层循环输出一次 `ab`，外层循环 3 次，最终结果是 `ababab`。

第3题：嵌套循环与 break 的局部作用

以下程序的输出结果是：

```
for i in "CHINA":
    for k in range(2):
        print(i, end="")
        if i == "N":
            break
```

参考答案：CCHHIINAA

解析：

1. 外层循环依次取字符串 `CHINA` 中的字符：`C`、`H`、`I`、`N`、`A`。内层循环 `range(2)` 正常情况下会执行 2 次，所以每个字符通常会打印两遍。
 2. 当外层字符是 `C`、`H`、`I`、`A` 时，条件 `i == "N"` 不成立，内层循环完整执行 2 次，因此分别输出 `CC`、`HH`、`II`、`AA`。
 3. 当外层字符是 `N` 时，先执行 `print(i, end="")` 输出一个 `N`，然后条件 `i == "N"` 成立，执行 `break` 跳出内层循环，因此 `N` 只输出一次。
 4. 把各部分连起来就是 `CCHHIINAA`。
- 非常关键的点：遇到 `N` 时，第一次打印 `N` 后，执行 `break`，`break` 只停止内层循环，一共有两层循环。只是不让 `N` 打印第二次。但是外层循环还没停，外层循环还会继续往后取下一个字符 `A`。

第4题：遍历字符串时遇到指定字符就停止

以下程序的输出结果是：

```
for i in "the number changes":
    if i == "n":
        break
    else:
        print(i, end=" ")
```

参考答案：the （e 后面还有一个空格）

解析：

1. 程序从左到右遍历字符串 `the number changes`。在遇到字符 `n` 之前，都会执行 `else` 中的输出语句。
2. 前几个字符依次是 `t`、`h`、`e`、空格，前三个字母和空格都会被输出。接下来遇到 `n`，条件 `i == "n"` 成立，执行 `break`，循环立刻停止。
3. 所以实际输出是 `the` 后面带一个空格。末尾的空格不容易看出来,但理解时要知道空格确实被输出过。

注意点：第三题也有`break`不过情况大有不同，需要做好区分

第5题：条件表达式与字符串大小比较

以下程序的输出结果是：

```
t = "Python"
print(t if t >= "python" else "None")
```

参考答案：None

解析：

1. 这里要判断 `t >= "python"`，需要比较字符串的大小。
2. 字符串比较按字符的 Unicode 编码顺序逐位比较。第一个字符就能决定结果：大写 `P` 的编码小于小写 `p`。
3. 因此 `"Python" >= "python"` 不成立，条件表达式选择 `else` 后面的值，输出 `None`。注意这里的 `None` 是字符串内容，不是空值对象。

关键点：字符串大小：数字 < 大写字母 < 小写字母